

Implementing the XOR gate through evolutionary algorithms

Pietro Malagoli

March 31, 2025

The aim of this project is to implement a simple logic gate, the XOR gate, on a basic neural network, and to do so by means of evolutionary algorithms. This work is mostly propaedeutic for the following project, which would be to use evolutionary algorithm to study the evolution of the basic computational cell, in order to verify if the neuron (i.e. a cell dedicated to computation, characterized by a threshold mechanic and a polarity) is the only possible solution achievable through evolution or if, instead, there are other possibilities which haven't been explored.

First of all, I'll briefly explain how the code is structured. It is divided between two files:

- *network.py* contains:
 - class *Network*: implements the network structure, with its basic attributes and methods.
 - function *evolution(par)*: it's the evolutionary algorithm that searches the parameter space for the best network that implements the target gate.
- *xor.ipynb* (this file): here the simulation parameters are defined, the code is executed, and the resulting data is briefly analyzed.

Now I'll present the Network class: this class is used to construct a basic neural network object, which in this project acts as the individual of the population that undergoes evolution. Each Network object is characterized by some attributes:

- 2 input neurons, 1 output neuron.
- S intra neurons, organized in a single layer.
- An array Theta, containing the threshold values of each neuron.
- A weight matrix J. The element J_{ij} refers to the link from neuron j to neuron i.

- A connectivity matrix C, which expresses the presence or absence of a link between two given neurons. It uses the same index convention as J.
- A fitness value.
- An activation function, which is common to every Network object.

Note that both matrices J and C are symmetric, but since the links in the network are always forward-directioned, I actually consider just the lower triangle in both matrices. In the evolution algorithm, I also update the higher triangle elements, but only for simplicity, since it's never considered. The algorithm would surely benefit in terms of performance if the two matrices were always kept symmetric.

Next, I'll briefly explain the method `Network.generate(par)`, which is used to generate a random Network object: this method randomly sets all the parameters mentioned above (except for the activation function, which is fixed). All the parameters are uniformly sampled from the respective range and the neurons are all set to zero. Moreover, the fitness of the network is computed, using the appropriate method.

The search algorithm selects the individuals based on their fitness value, which is defined as such:

$$f(\Omega) = \frac{1}{N_I N} \sum_k^N \left[\sum_I^{N_I} \alpha (O^T - O_k)_I^2 + S \left(\frac{\sum_{i,j} C_{i,j}}{2} \right)_I \right] \quad (1)$$

where Ω is the set of individuals, N_I is the number of possible inputs to the network, in this case 4, S is the number of intra neurons, and N is the number of individuals. O^T is the target output, while O_k is the actual output, given input I. Finally, α is a factor used to weight the first term of the fitness function and in this case was set to 50.

Note: even though the fitness function is named as such, the objective is to minimize it. It would maybe be more correct to call it 'error' function.

Lastly, I'll quickly present the evolutionary algorithm implemented:

1. **Generation:** Generate N individuals, called 'solutions'.
2. **Evaluation:** Evaluate the fitness value of each one of those solutions.
3. **Selection:** Discard the solutions whose fitness is *above* the mean fitness of the whole set of solutions.
4. **Extraction of parents:** Randomly choose n parents from the remaining to generate offspring from, where n is set by the parameter 'mutation_ratio'.
5. **Mutation:** Generate for each parent an offspring by mutation of the parameters S, J, C, and Theta.
6. **Random Generation:** Generate the remaining individuals to get back to the original population at point 2 by random generation.

7. Back to step 2.

A couple of comments about this procedure:

- the mutations to each parameter are either sampled from a gaussian distribution, having mean = 0 and standard deviation = 'Theta.mutation.radius' or 'J.mutation.radius', or from a uniform distribution for C and finally for S by sampling from the array [-1,0,+1] with respective probabilities of $p=[0.2,0.6,0.2]$.
- At the end of the iteration, the new solutions set is formed by:
 - original selected individuals whose fitness whose equal to or below average.
 - offspring generated by mutation from parents extracted from the above individuals.
 - randomly generated individuals.

Next, the mean fitness value is plotted against the iteration in Figure 1. As can also be understood from the above output, the algorithm quickly (shortly before 100 iterations) reaches a low fitness value. Then, if no early stopping is implemented, it oscillates around the solution; such oscillations are only due to the second term of the fitness function (i.e. the cost). In Figure 2 is shown an example of some parameters of a solution found in the current run, such as fitness value, number of intra neurons (S), and produced output for each possible input.

In Figure 3 , the connectivity matrix of Solution #0 is presented.

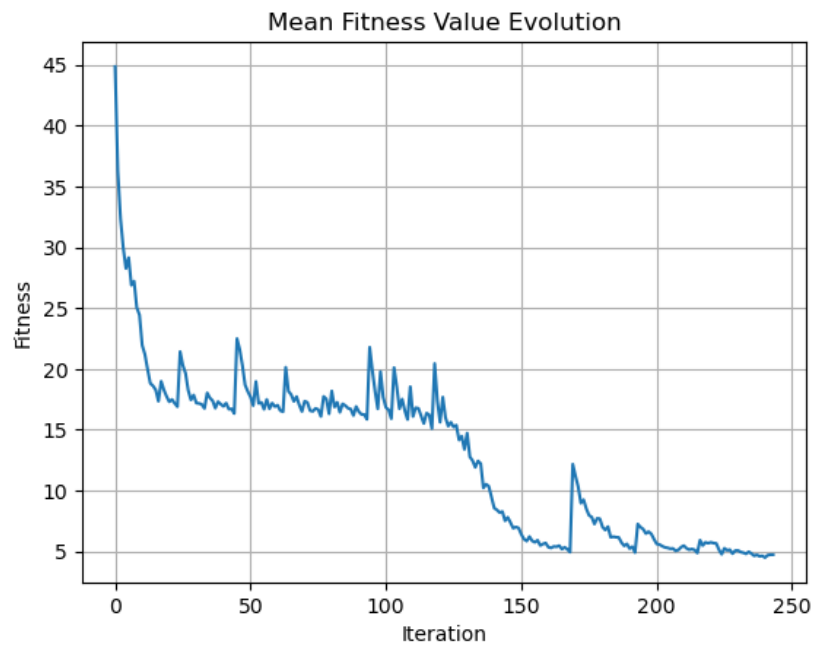


Figure 1: Mean fitness evolution (here convergence was reached after 244 iterations, with a final mean fitness of 4.74).

```
Solution #0...  
Fitness: 4.5  
S: 1  
[0, 0] Output: 0.0  
[0, 1] Output: 1.0  
[1, 0] Output: 1.0  
[1, 1] Output: 0.0
```

Figure 2: Example of a solution from the current run.

```
Solution #0...  
[[0. 0. 0. 1.]  
 [0. 0. 1. 1.]  
 [1. 1. 0. 1.]  
 [1. 1. 1. 0.]]
```

Figure 3: Connectivity matrix of Solution #0. It only matters the lower triangle of the matrix.